

PRESIDENCY UNIVERSITY

CSE324: Database Management Systems Laboratory

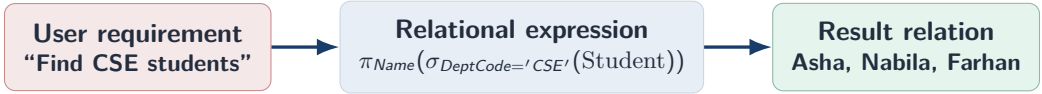
Class 02: Relational Algebra & Relational Calculus

From English Questions to Correct Database Queries

Afzal Hossan
Lecturer
Department of Computer Science and Engineering
Presidency University

Today: From Relations to Queries

THE BIG QUESTION How do we describe precisely which rows and columns a user wants?



Relational algebra

An **operational** description: select rows, keep columns, combine relations.

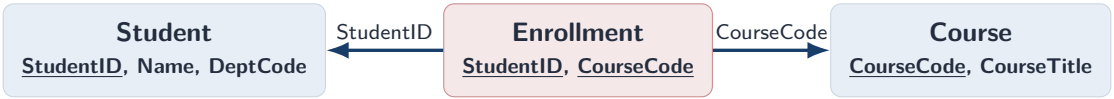
Relational calculus

A **condition-based** description: specify which tuples satisfy the requirement.

Practical target: write a correct expression and *calculate its result* on real small tables.

Our Running Database: Three Relations

Use the same fictional Presidency University registration database for every exercise.



Why Enrollment?

One student can register for multiple courses, and each course can have many students.

Our task

Combine the three relations when the requested answer contains attributes from several tables.

Underlined attributes indicate the primary key(s). Enrollment uses a composite primary key.

Data Sheet A: Student and Course

Student

6 tuples

StudentID	Name	DeptCode
101	Asha	CSE
102	Rahim	EEE
103	Nabila	CSE
104	Tanvir	BBA
105	Farhan	CSE
106	Mitu	EEE

Course

5 tuples

CourseCode	CourseTitle
CSE323	DBMS Theory
CSE211	Data Structures
CSE324	DBMS Lab
EEE201	Circuits
MAT101	Mathematics

Keep this data sheet: Asha, Nabila and Farhan are in CSE; Mitu has no enrollment.

Data Sheet B: Enrollment

StudentID	CourseCode
101	CSE323
101	CSE211
102	EEE201
103	CSE323
103	CSE211
104	MAT101
105	CSE211

Three observations

1. Student 106 (Mitu) is not listed.
2. Course CSE324 is not listed.
3. Students 101 and 103 take both CSE323 and CSE211.

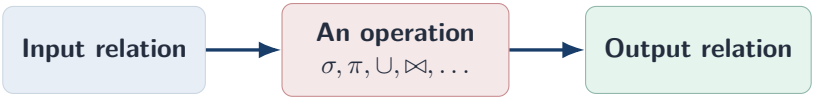
Why these examples matter

We can use them for **joins**, **set operations**, and **"not enrolled"** questions.

What Is Relational Algebra?

First understand selection and projection.

A formal language for building new relations from existing relations.



Closure principle

The result of each relational algebra operation is itself a relation, so operations can be **nested** or combined.

Learn by doing

For every operator: name the input, perform the operation, and display the exact output schema and tuples.

Selection σ: Keep Matching Rows

Question: Show complete records of students in the CSE department.

$$\sigma_{DeptCode='CSE'}(\text{Student})$$

Input: Student

ID	Name	Dept
101	Asha	CSE
102	Rahim	EEE
103	Nabila	CSE
104	Tanvir	BBA
105	Farhan	CSE
106	Mitu	EEE

Output: only matching rows

ID	Name	Dept
101	Asha	CSE
103	Nabila	CSE
105	Farhan	CSE

Remember

σ changes the **number of rows**, not the set of columns.

Selection Conditions: Comparison and Logic

Common comparison operators

$$= \quad \neq \quad < \quad \leq \quad > \quad \geq$$

Combine simple conditions

$$\sigma_{StudentID>103}(\text{Student})$$

$$\sigma_{DeptCode='CSE' \ \wedge \ StudentID>102}(\text{Student})$$

$$\sigma_{DeptCode='CSE' \ \vee \ DeptCode='EEE'}(\text{Student})$$

Predict!

1. IDs greater than 103?
2. CSE students with ID above 102?
3. Students in CSE or EEE?

Boolean conditions

$\wedge$: AND
$\vee$: OR
$\neg$: NOT

Answers: (1) 104,105,106; (2) 103,105; (3) 101,102,103,105,106.

Projection π: Keep Selected Columns

Question: Display just the IDs and names of all students.

$$\pi_{StudentID,Name}(\text{Student})$$

Input has three columns

ID	Name	Dept
101	Asha	CSE
102	Rahim	EEE
103	Nabila	CSE
104	Tanvir	BBA
105	Farhan	CSE
106	Mitu	EEE

Output has two columns

StudentID	Name
101	Asha
102	Rahim
103	Nabila
104	Tanvir
105	Farhan
106	Mitu

π chooses **columns**. In classical relational algebra, duplicate result tuples are eliminated.

Projection Removes Duplicate Tuples

Question: Which distinct department codes exist?

$$\pi_{DeptCode}(\text{Student})$$

Before projection

DeptCode values: CSE, EEE, CSE, BBA, CSE, EEE

After projection

DeptCode
CSE
EEE
BBA

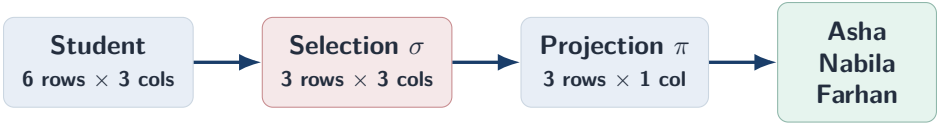
Important distinction from SQL

Relational algebra returns set-valued results; SQL can retain duplicates unless `DISTINCT` is used.

Combine Selection and Projection

Question: Find the *names* of students who belong to CSE.

$$\pi_{Name}\Big(\sigma_{DeptCode='CSE'}(\text{Student})\Big)$$



Evaluation rule: Work *inside out*. First select the CSE rows, then retain the Name attribute.

Set Operations: Core Idea

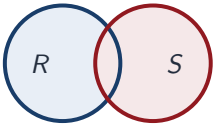
Comparing compatible sets of tuples.

Operations

$R \cup S$	Union: in R or S (or both)
$R \cap S$	Intersection: in both
$R - S$	Difference: in R but not S

Key prerequisite

Relations must be **union-compatible**: same number of attributes and compatible corresponding domains.



Two overlapping sets of tuples

Union Compatibility: Check Before You Combine

Valid

$A = \pi_{StudentID}(\sigma_{CourseCode='CSE323'}(\text{Enrollment}))$

$B = \pi_{StudentID}(\sigma_{CourseCode='CSE211'}(\text{Enrollment}))$

$A \cup B$ **valid**: both contain one StudentID attribute.

Invalid

$\text{Student} \cup \text{Enrollment}$

Not valid: Student has three attributes; Enrollment has two.

Repair? Project corresponding compatible attributes first.

Check all three: same number of columns, compatible value domains, and aligned attribute meanings/names (rename where needed).

Define Two Sets of Student IDs

We will reuse A and B for union, intersection, and difference.

Set A: registered for CSE323

$A = \pi_{StudentID}(\sigma_{CourseCode='CSE323'}(\text{Enrollment}))$

$\{101, 103\}$

Set B: registered for CSE211

$B = \pi_{StudentID}(\sigma_{CourseCode='CSE211'}(\text{Enrollment}))$

$\{101, 103, 105\}$

Question: Who takes either course? Both courses? Only CSE211?

Union $\cup$: In Either Set

Question: Which students are enrolled in CSE323 *or* CSE211?

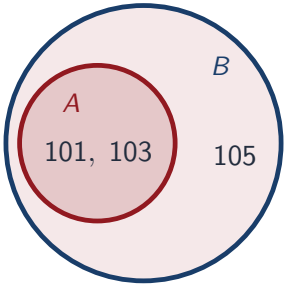
The two enrollment sets

A **(CSE323)** $= \{101, 103\}$

B **(CSE211)** $= \{101, 103, 105\}$

Result: Inclusive OR

$$A \cup B = \{101, 103, 105\}$$



Entire shaded circle is $A \cup B$

Notice: All members of A are in B.
Therefore, $A \subset B$ and $A \cup B = B$.

Intersection ∩: In Both Sets

Question: Which students are registered in both CSE323 and CSE211?

The two enrollment sets

A **(CSE323)** $= \{101, 103\}$

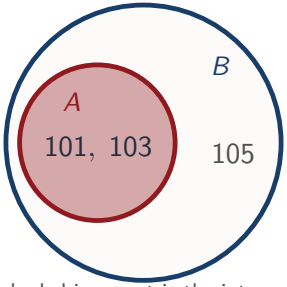
B **(CSE211)** $= \{101, 103, 105\}$

Intersection result

$$A \cap B = \{101, 103\}$$

Observation: Every element of A is already inside B.

So, because $A \subset B$, the intersection is simply A.



The shaded inner set is the intersection because those elements belong to both sets.

Difference −: The Order Matters

Question: Which students take CSE211 but *not* CSE323?

Given sets

A **(CSE323)** $= \{101, 103\}$

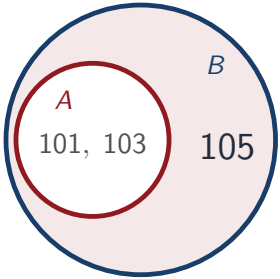
B **(CSE211)** $= \{101, 103, 105\}$

Main result

$$B - A = \{105\}$$

Reverse the order

$$A - B = \varnothing$$



Shaded region = elements in B but not in A

Cartesian Product $\times$: All Pairings

How can information from multiple relations be linked?

Pair every tuple of R with every tuple of S. Here only three of the five resulting attributes are displayed.

Student $\times$ Course

Record count

$$6 \times 5 = 30$$

tuples

StudentID	Name	CourseCode
101	Asha	CSE323
101	Asha	CSE211
101	Asha	CSE324
101	Asha	EEE201
101	Asha	MAT101
... 25 more tuple pairings ...		

General rule: if $|R| = m$ and $|S| = n$, then $|R \times S| = mn$.

Theta Join and Equijoin

A theta join is a Cartesian product followed by a selection condition.

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

Join Student with Enrollment

$$\text{Student} \bowtie_{\text{Student.}\textit{StudentID}=\text{Enrollment.}\textit{StudentID}} \text{Enrollment}$$

- θ can use comparisons such as $=$, $<$ or $>$.
- An **equijoin** uses equality ($=$) as its condition.
- Both copies of the join attribute can remain in a theta/equijoin result unless explicitly projected away.

Natural Join ⋈

Join on all attributes with the same names and retain only one copy of each common attribute.

Student ⋈ Enrollment

What is matched?

- Shared attribute: `StudentID`.
- Rows match when their StudentID values are equal.
- Only one StudentID column appears in the natural-join result.
- Result contains 7 matching enrollment rows.

ID	Name	Dept	Course
101	Asha	CSE	CSE323
101	Asha	CSE	CSE211
102	Rahim	EEE	EEE201
103	Nabila	CSE	CSE323
103	Nabila	CSE	CSE211
104	Tanvir	BBA	MAT101
105	Farhan	CSE	CSE211

Natural Join: The Hidden Danger

Natural join matches *every* identically named attribute, not just the intended one.

Safe in our example

Student(StudentID, Name, DeptCode)
Enrollment(StudentID, CourseCode)

Only StudentID is shared.

Possible surprise

If two relations share a column name that does *not* mean the same thing, natural join adds an unintended equality condition.

Use an **explicit conditional join** when clarity is needed.

Also note: A natural join over relations with no common attribute names behaves like a Cartesian product.

Rename ρ: Give a Relation a New Name

Aliases make repeated uses of a relation possible.

$$\rho_{S_1}(\text{Student}) \qquad \rho_{S_2}(\text{Student})$$

Meaning

Two **separate references** to the same Student relation: one called S_1, the other S_2.

Why useful?

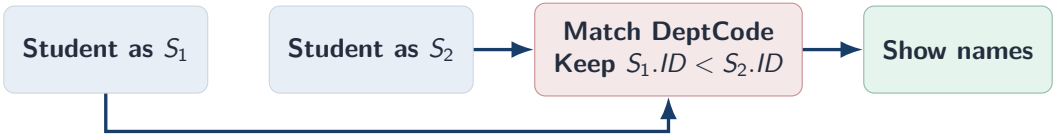
Compare a student with another student, e.g., to find students in the **same department**.

Use qualified attributes such as $S_1.StudentID$ and $S_2.StudentID$ to avoid ambiguity.

Self-Join: Find Same-Department Pairs

Requirement: Find two *different* students in the same department.

$$\pi_{S_1.Name,\, S_2.Name}\left(\sigma_{\substack{S_1.DeptCode=S_2.DeptCode \\ S_1.StudentID<S_2.StudentID}}\left(\rho_{S_1}(\text{Student}) \times \rho_{S_2}(\text{Student})\right)\right)$$



- $S_1.DeptCode = S_2.DeptCode$ ensures same department.
- $S_1.StudentID < S_2.StudentID$ prevents self-pairs and reversed duplicates.

Self-Join: Verify the Four Result Pairs

Student 1	Student 2	Dept
Asha	Nabila	CSE
Asha	Farhan	CSE
Nabila	Farhan	CSE
Rahim	Mitu	EEE

Why no other pairs?

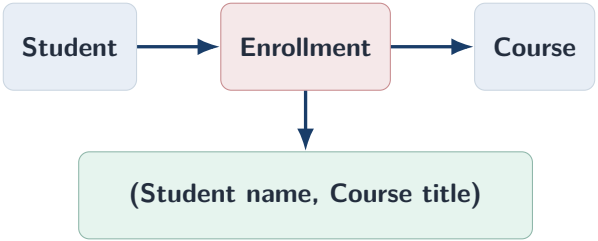
- Tanvir is the only BBA student.
- The three CSE students produce $\binom{3}{2} = 3$ distinct pairs.
- The two EEE students produce $\binom{2}{2} = 1$ pair.

The **same-relation self-join** is the main reason to introduce rename today.

Three-Relation Query: Meaningful Information

Requirement: Show each registered student and the title of the registered course.

$$\pi_{Name,CourseTitle}(\text{Student} \bowtie \text{Enrollment} \bowtie \text{Course})$$



Natural joins match StudentID in the first join and CourseCode in the second.

Three-Relation Query: Complete Result

Student Name	Course Title
Asha	DBMS Theory
Asha	Data Structures
Rahim	Circuits
Nabila	DBMS Theory
Nabila	Data Structures
Tanvir	Mathematics
Farhan	Data Structures

Check the output

- Exactly **7** tuples.
- Each row is a genuine enrollment.
- Mitu is absent.
- DBMS Lab is absent.

Sanity-check tip: Count the matching tuples before writing down the final result.

Queries Involving "No Matching Records"

Question: Find student IDs that never appear in Enrollment.

$$\pi_{StudentID}(\text{Student}) - \pi_{StudentID}(\text{Enrollment})$$

All students

$\{101, 102, 103, 104, 105, 106\}$

Enrolled students

$\{101, 102, 103, 104, 105\}$

$$\{106\} \Longrightarrow \textbf{Mitu}$$

To obtain Name rather than StudentID, join the difference result with Student and project Name.

Relational Calculus: Describe the Result

Introductory coverage: meaning, notation, logical predicates.

Relational algebra

HOW

"Select CSE rows; then project Name."

Relational calculus

WHAT

"Find tuples representing CSE students."

$$\{t \mid P(t)\}$$

- t: a tuple variable; $P(t)$: a condition/predicate.
- The answer consists of those tuples for which the predicate is true.

Tuple Relational Calculus: Read the Symbols

$$\{s \mid \text{Student}(s) \wedge s.\textit{DeptCode} = \text{'CSE'}\}$$

Read piece by piece

- s: variable representing a tuple.
- $\text{Student}(s)$: tuple s belongs to Student.
- $s.\textit{DeptCode} = \text{'CSE'}$: CSE condition.
- $\wedge$: both statements must hold.

Result

Three **complete Student tuples**:

(101, Asha, CSE)
(103, Nabila, CSE)
(105, Farhan, CSE)

This expression returns full tuple values; it does not return just names.

Logical Connectives and Quantifiers

Logic you will use today

$P \wedge Q$	both P and Q
$P \vee Q$	P or Q (inclusive)
$\neg P$	not P
$\exists e\, P(e)$	there exists some e
$\forall e\, P(e)$	for every e

Think in questions

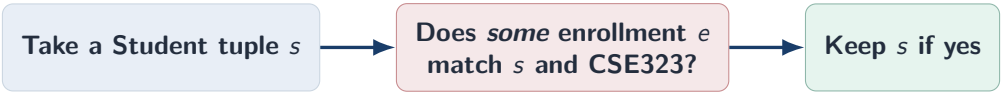
- **AND**: in CSE and ID > 102.
- **OR**: in CSE or EEE.
- **EXISTS**: has at least one enrollment.
- **NOT EXISTS**: has no enrollment.

We will **recognize** $\forall$ but will not build complicated nested universal-quantifier queries in this 1-credit lab.

TRC with ∃: Registered for CSE323

Question: Which Student tuples have a matching enrollment in CSE323?

$$\left\{ s \;\middle|\; \text{Student}(s) \;\wedge\; \exists e\big(\text{Enrollment}(e) \wedge e.\mathit{StudentID} = s.\mathit{StudentID} \;\wedge\; e.\mathit{CourseCode} = \text{'CSE323'}\big) \right\}$$



Correct answer

Complete Student tuples for **Asha and Nabila**.

TRC with ¬∃: No Enrollment

Question: Which Student tuples have no matching Enrollment tuple?

$$\{s \mid \text{Student}(s) \wedge \neg\exists e\,(\text{Enrollment}(e) \wedge e.\mathit{StudentID} = s.\mathit{StudentID})\}$$

Interpretation

"Return students s for which **no** enrollment e has the same StudentID."

Result

(106, Mitu, EEE)

Connection: The same idea can be expressed in relational algebra using set difference.

Tuple Calculus vs Domain Calculus

Both are declarative; today the emphasis is on understanding tuple calculus.

Tuple Relational Calculus (TRC)

Variables represent **entire tuples**.

Example variable: s from Student.

$\{s \mid \text{Student}(s) \wedge s.\mathit{DeptCode} = \text{'CSE'}\}$

Domain Relational Calculus (DRC)

Variables represent **individual attribute values**.

Illustration: $(\mathit{id}, \mathit{name}, \mathit{dept})$ for one Student tuple.

Recognition only; detailed DRC expressions are out of today's scope.

Safety idea: Restrict variables to the database relations and use well-formed conditions, so requested results are meaningful and finite.

One Requirement, Three Ways to Describe It

Preview only: SQL is taught systematically in Lecture 3.

Requirement: Find the names of CSE students

Relational algebra

$$\pi_{Name}(\sigma_{DeptCode='CSE'}(\text{Student}))$$

Tuple relational calculus

$$\{t \mid \exists s(\text{Student}(s) \land s.DeptCode = \text{'CSE'} \land t.Name = s.Name)\}$$

Output tuple t contains the single attribute Name.

Equivalent SQL for the requested names

```
SELECT DISTINCT Name FROM Student WHERE DeptCode = 'CSE';
```

SQL Preview: Join and Project

Question: Find the names of students registered in CSE323.

Relational algebra

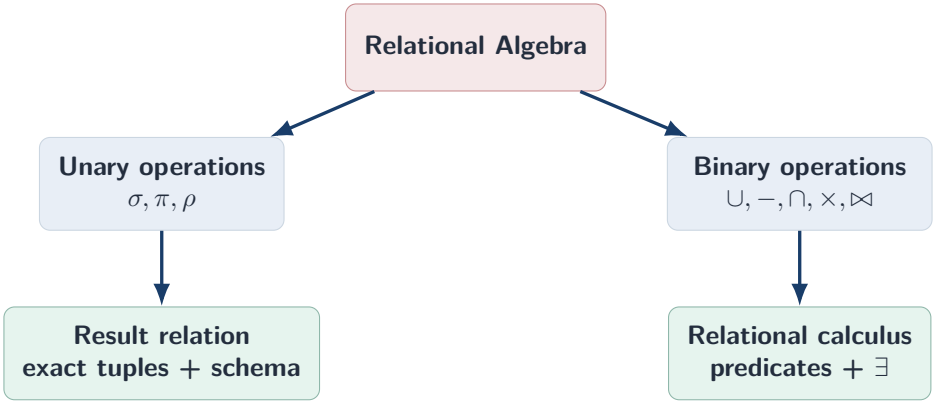
$$\pi_{Name}(\text{Student} \bowtie \sigma_{CourseCode='CSE323'}(\text{Enrollment}))$$

```
SELECT DISTINCT s.Name
FROM Student AS s
JOIN Enrollment AS e
  ON s.StudentID = e.StudentID
WHERE e.CourseCode = 'CSE323';
```

Why DISTINCT?

Classical relational algebra removes duplicates; SQL normally allows duplicate output rows.

Lecture 2 Summary: Complete Operator Map



Understand the question → choose operations → verify the result.

Your Practical Lab: Relational Algebra

Use the Student, Course, and Enrollment data sheets from the beginning of this deck.

Core queries

1. Names of all CSE students.
2. Distinct department codes.
3. IDs of students with any enrollment.
4. Names of students without enrollment.
5. Course codes with no enrollment.

More complex queries

6. IDs enrolled in CSE323 or CSE211.
7. IDs enrolled in both courses.
8. IDs in CSE211 but not CSE323.
9. Student names with course titles (extension).
10. Same-department student pairs (extension).

Complete algebra 1–8 and both tuple-calculus tasks first. Then attempt 9–10 as extensions. Include result tables.

Your Practical Lab: Tuple Calculus & Submission

Tuple-calculus tasks

1. Write a TRC expression that returns **all EEE student tuples**; list the resulting tuples.
2. Write a TRC expression that returns **students who have no enrollment**; explain the role of $\neg\exists$.

Deliverable

L02_Relational_Queries.pdf

- Algebra tasks 1–8 (9–10: extension)
- Exact result tables or sets
- Two TRC expressions
- Explanation for one multistep query

First decide what the query means. Then choose the operation.